

SIHOS

Sistema de Gestión de Pacientes

Documentación Técnica — Punto 3

Lógica de Programación: CRUD de Pacientes

Laravel · PHP · MySQL · JavaScript

Nombres completos: Yeison Jobany Marín Chaguala

Cédula: 1110174030

Programa: Análisis y Desarrollo de Software (ADSO) — SENA

Prueba técnica desarrollador EST-SEM

1. Introducción

Este documento describe el desarrollo del Punto 3 de la prueba técnica: la construcción de una matriz CRUD (Crear, Leer, Actualizar, Eliminar) para la gestión de pacientes, cumpliendo los requerimientos de backend, frontend, integración y funcionalidades adicionales establecidos en el enunciado.

El proyecto fue desarrollado utilizando el framework Laravel 11 sobre PHP, con una base de datos relacional en MySQL, autenticación mediante sesiones (Laravel Breeze), y un frontend en Blade + JavaScript (Fetch API) que consume una API RESTful.

El código fuente completo se encuentra disponible en el repositorio de GitHub compartido junto con esta documentación.

2. Arquitectura y Stack Tecnológico

Capa	Tecnología	Justificación
Backend	Laravel 11 (PHP 8.2)	Framework MVC robusto, con Eloquent ORM, migraciones, validaciones y middlewares nativos.
Base de datos	MySQL	Motor relacional solicitado explícitamente en el enunciado.
Autenticación	Laravel Breeze (sesiones)	Cumple el requisito de 'autenticación mediante JWT o sesiones' de forma nativa y segura.
Frontend	Blade + Tailwind CSS + JavaScript	Vistas del lado del servidor combinadas con peticiones asíncronas (Fetch API).
Acceso a datos alternativo	PDO nativo de PHP	Capa adicional de acceso directo a MySQL, cumpliendo el requisito explícito de uso de PDO.
Pruebas	PHPUnit	Pruebas automatizadas de la API sobre una base de datos SQLite en memoria.

El proyecto sigue el patrón MVC (Modelo - Vista - Controlador), separando claramente la lógica de negocio (Modelos), la lógica de presentación (Vistas Blade) y la lógica de orquestación de peticiones (Controladores).

3. Diseño de la Base de Datos

Se diseñaron 5 tablas relacionadas mediante llaves foráneas, siguiendo la estructura solicitada en el enunciado:

Tabla	Campos principales	Relación
departamentos	id, nombre	Tabla padre de municipios

municipios	id, departamento_id, nombre	Pertenece a un departamento (belongsTo)
tipos_documento	id, nombre	Catálogo de tipos de documento
genero	id, nombre	Catálogo de géneros
pacientes	id, tipo_documento_id, numero_documento, nombre1, nombre2, apellido1, apellido2, genero_id, departamento_id, municipio_id, correo	Pertenece a tipo_documento, genero, departamento y municipio

3.1 Migración de la tabla pacientes

A continuación se muestra la migración más relevante, con sus 4 llaves foráneas y las validaciones de unicidad definidas a nivel de base de datos:

```
Schema::create('pacientes', function (Blueprint $table) {
    $table->id();
    $table->foreignId('tipo_documento_id')->constrained('tipos_documento')->onDelete('cascade');
    $table->string('numero_documento')->unique();
    $table->string('nombre1');
    $table->string('nombre2')->nullable();
    $table->string('apellido1');
    $table->string('apellido2')->nullable();
    $table->foreignId('genero_id')->constrained('genero')->onDelete('cascade');
    $table->foreignId('departamento_id')->constrained('departamentos')->onDelete('cascade');
    $table->foreignId('municipio_id')->constrained('municipios')->onDelete('cascade');
    $table->string('correo')->unique();
    $table->timestamps();
});
```

Las columnas numero_documento y correo se definieron como unique() para evitar duplicados a nivel de base de datos, reforzando la validación que también se realiza en el backend. Las columnas nombre2 y apellido2 se definieron como nullable() dado que no todos los pacientes tienen segundo nombre o segundo apellido.

3.2 Seeders (datos de prueba)

Se implementaron seeders para poblar automáticamente la base de datos, cumpliendo los volúmenes de datos solicitados:

- Departamentos: 5 registros (Tolima, Cundinamarca, Antioquia, Valle del Cauca, Atlántico)
- Municipios: 2 registros por cada departamento (10 en total)
- Tipos de documento: 2 registros (Cédula de Ciudadanía, Tarjeta de Identidad)
- Género: 2 registros (Masculino, Femenino)
- Usuario administrador: admin@sihos.com, contraseña encriptada con bcrypt
- Pacientes: 5 registros de prueba

Los seeders de municipios y pacientes buscan dinámicamente los IDs de sus tablas padre por nombre (en lugar de asumir IDs fijos), lo cual hace el proceso de siembra más robusto ante cambios de orden:

```
$tolima = DB::table('departamentos')->where('nombre', 'Tolima')->value('id');

DB::table('municipios')->insert([
    ['departamento_id' => $tolima, 'nombre' => 'Ibagué'],
    ['departamento_id' => $tolima, 'nombre' => 'Espinal'],
    // ...
]);
```

4. Modelos Eloquent y Relaciones

Cada tabla cuenta con su modelo Eloquent correspondiente. El modelo Paciente centraliza las 4 relaciones belongsTo hacia sus tablas de catálogo:

```
class Paciente extends Model
{
    protected $fillable = [
        'tipo_documento_id', 'numero_documento', 'nombre1', 'nombre2',
        'apellido1', 'apellido2', 'genero_id', 'departamento_id',
        'municipio_id', 'correo',
    ];

    public function tipoDocumento() { return $this->belongsTo(TipoDocumento::class); }
    public function genero()        { return $this->belongsTo(Genero::class); }
    public function departamento() { return $this->belongsTo(Departamento::class); }
    public function municipio()     { return $this->belongsTo(Municipio::class); }
}
```

La propiedad \$fillable restringe explícitamente qué columnas pueden asignarse de forma masiva (mass assignment), como medida de seguridad estándar de Laravel.

5. Autenticación

Se implementó autenticación mediante sesiones utilizando Laravel Breeze, que provee registro, inicio de sesión, recuperación de contraseña y verificación de correo de forma nativa. Todas las rutas del módulo de pacientes están protegidas con el middleware auth, de modo que solo usuarios autenticados pueden acceder a la información de pacientes:

```
Route::middleware(['auth'])->group(function () {
    Route::get('/pacientes-view', fn () => view('pacientes.index'))->name('pacientes.view');
    Route::apiResource('pacientes', PacienteController::class)->except(['create', 'edit']);
});
```

Se optó por autenticación basada en sesiones (en lugar de JWT) porque el frontend y el backend residen en la misma aplicación Laravel, lo cual permite aprovechar la protección CSRF nativa del framework sin necesidad de gestionar tokens manualmente.

6. API RESTful — PacienteController

El controlador PacienteController expone las operaciones CRUD siguiendo la convención RESTful de Laravel:

Método HTTP	Ruta	Acción	Descripción
-------------	------	--------	-------------

GET	/pacientes	index()	Lista pacientes con paginación y búsqueda opcional
POST	/pacientes	store()	Crea un nuevo paciente (validado)
GET	/pacientes/{id}	show()	Obtiene un paciente específico
PUT	/pacientes/{id}	update()	Actualiza un paciente existente (validado)
DELETE	/pacientes/{id}	destroy()	Elimina un paciente

6.1 Validación de datos

Cada operación de escritura valida los datos de entrada utilizando el sistema de validación de Laravel, cumpliendo el requisito de 'validación de correo electrónico' y protección de integridad de datos:

```
$validated = $request->validate([
    'tipo_documento_id' => 'required|exists:tipos_documento,id',
    'numero_documento' => 'required|string|max:20|unique:pacientes,numero_documento',
    'nombre1'           => 'required|string|max:100',
    'nombre2'           => 'nullable|string|max:100',
    'apellido1'         => 'required|string|max:100',
    'apellido2'         => 'nullable|string|max:100',
    'genero_id'         => 'required|exists:genero,id',
    'departamento_id'  => 'required|exists:departamentos,id',
    'municipio_id'     => 'required|exists:municipios,id',
    'correo'            => 'required|email|unique:pacientes,correo',
]);
```

La regla exists:tabla,columna evita que se guarden llaves foráneas inexistentes, mientras que unique evita duplicidad de documento y correo. En las actualizaciones (update), la regla unique excluye el propio registro mediante su ID, para que un paciente pueda conservar su documento/correo sin generar un falso conflicto.

6.2 Búsqueda y paginación

El método index() implementa búsqueda dinámica por nombre, apellido o correo, y pagina los resultados de 5 en 5:

```
public function index(Request $request)
{
    $query = Paciente::with(['tipoDocumento', 'genero', 'departamento', 'municipio']);

    if ($request->filled('buscar')) {
        $buscar = $request->input('buscar');
        $query->where(function ($q) use ($buscar) {
            $q->where('nombre1', 'like', "%{$buscar}%")
                ->orWhere('nombre2', 'like', "%{$buscar}%")
                ->orWhere('apellido1', 'like', "%{$buscar}%")
                ->orWhere('apellido2', 'like', "%{$buscar}%")
                ->orWhere('correo', 'like', "%{$buscar}%");
        });
    }
}
```

```
    return response()->json($query->orderBy('id','desc')->paginate(5));
}
```

El método `with(...)` realiza eager loading de las 4 relaciones del paciente, evitando el problema de N+1 consultas y optimizando el rendimiento de la API.

7. Capa de Acceso a Datos con PDO

Además del acceso a datos mediante Eloquent (que internamente utiliza PDO), se implementó un endpoint adicional que emplea PDO de forma explícita y directa, cumpliendo el requisito puntual del enunciado. Este endpoint genera un reporte de cantidad de pacientes por departamento:

```
$pdo = new PDO(
    "mysql:host={$host};port={$puerto};dbname={$baseDatos};charset=utf8mb4",
    $usuario, $password,
    [PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION]
);

$sql = "SELECT d.nombre AS departamento, COUNT(p.id) AS total_pacientes",
    . " FROM departamentos d",
    . " LEFT JOIN pacientes p ON p.departamento_id = d.id",
    . " GROUP BY d.id, d.nombre",
    . " ORDER BY total_pacientes DESC";

$consulta = $pdo->prepare($sql);
$consulta->execute();
$resultados = $consulta->fetchAll(PDO::FETCH_ASSOC);
```

Se utilizó una sentencia preparada (`prepare/execute`) y `PDO::ERRMODE_EXCEPTION` para el manejo controlado de errores, siguiendo buenas prácticas contra inyección SQL. Ruta disponible: `GET /reportes/pacientes-por-departamento`.

8. Frontend (HTML, CSS, JavaScript)

La interfaz de usuario se construyó como una vista Blade (`pacientes/index.blade.php`) que incluye un formulario para crear/editar pacientes, una tabla de listado, un campo de búsqueda y controles de paginación, todo estilizado con Tailwind CSS.

8.1 Comunicación con la API (fetch)

Todas las operaciones (crear, listar, editar, eliminar) se realizan mediante peticiones asíncronas con la Fetch API, sin recargar la página, incluyendo el token CSRF requerido por Laravel para peticiones que modifican datos:

```
const respuesta = await fetch(url, {
    method: metodo, // POST o PUT según si se crea o edita
    headers: {
        'Content-Type': 'application/json',
        'X-CSRF-TOKEN': document.querySelector('meta[name="csrf-token"]').content,
    },
    body: JSON.stringify(datos),
});

const resultado = await respuesta.json();
```

```

if (!respuesta.ok) {
    mostrarMensaje('Error: ' + Object.values(resultado.errors).flat().join(' | '), 'error');
    return;
}
mostrarMensaje(resultado.message, 'exito');

```

El manejo de errores muestra al usuario los mensajes devueltos por las validaciones del backend (por ejemplo, formato de correo inválido o documento duplicado), cumpliendo el requisito de manejo de errores en el frontend.

8.2 Selects dependientes (Departamento → Municipio)

El formulario implementa un patrón de selects encadenados: al elegir un departamento, se consulta dinámicamente la lista de municipios que le pertenecen, evitando mostrar combinaciones inválidas:

```

document.getElementById('departamento_id').addEventListener('change', async (e) => {
    const municipios = await fetch(`/municipios/${e.target.value}`).then(r => r.json());
    llenarSelect('municipio_id', municipios);
});

```

8.3 Búsqueda y paginación en el frontend

El input de búsqueda dispara la recarga de la tabla en cada pulsación de tecla, y la paginación se renderiza dinámicamente según el total de páginas devuelto por la API:

```

document.getElementById('buscar').addEventListener('input', (e) => {
    terminoBusqueda = e.target.value;
    paginaActual = 1;
    cargarPacientes();
});

```

9. Pruebas Unitarias (PHPUnit)

Se desarrollaron 7 pruebas de tipo Feature en tests/Feature/PacienteApiTest.php, ejecutadas sobre una base de datos SQLite en memoria (aislada de la base de datos de producción) mediante el trait RefreshDatabase:

Prueba	Qué valida
puede_crear_un_paciente_con_datos_validos	Creación exitosa (HTTP 201) y persistencia en base de datos
rechaza_la_creacion_si_el_correo_es_invalido	Validación de formato de correo (HTTP 422)
rechaza_la_creacion_si_faltan_campos_obligatorios	Validación de campos requeridos
puede_listar_pacientes	Listado correcto vía GET /pacientes
puede_actualizar_un_paciente	Actualización correcta vía PUT
puede_eliminar_un_paciente	Eliminación correcta vía DELETE
un_usuario_no_autenticado_no_puede_acceder_a_pacientes	Protección de rutas (HTTP 401 sin sesión)

Resultado de la ejecución (comando `php artisan test`): 32 pruebas pasadas (87 aserciones), incluyendo las 7 propias del módulo de pacientes y las 25 que Laravel Breeze provee por defecto para autenticación y perfil de usuario.

10. Conclusión

El desarrollo cumple con los requerimientos del Punto 3 de la prueba técnica: CRUD funcional de pacientes con base de datos relacional (MySQL), migraciones y seeders completos, API RESTful validada y segura, autenticación por sesiones, capa de acceso con PDO, interfaz interactiva con Fetch API, búsqueda, paginación y pruebas automatizadas con PHPUnit. El código fuente completo, junto con las instrucciones de instalación, se encuentra disponible en el repositorio de GitHub compartido.